NetLift: A Causal Decision System for Incremental Profit Optimization

Phase 3: Requirements Gathering

Digital Egypt Pioneers Initiative (DEPI), Microsoft Machine Learning Engineer Track

October 2026

Team leader: Mahmoud Nadeem

Team member	Role
Mahmoud Nadeem	Team Leader; Advanced Models and Experiment Tracking
Aliaa Gomaa	Data Engineering and Experiment Validity
Mohammed Sherif	Uplift Modelling Core and Dashboard
Ali Alaa	Evaluation, Statistics and Decision Layer
Mazen Mahmoud	MLOps, Backend and Deployment

Repository: https://github.com/mahmoud-nadeem/netlift

Contents

1. Stakeholder Analysis
2. User Stories and Use Cases
3. Functional Requirements
4. Non-functional Requirements

1. Stakeholder Analysis

Author: Mohammed Sherif · Task: T31

NetLift has no live business users this semester: both datasets are historical, public and de-identified, and the “campaign” is a simulated replay over a held-out split. The five business stakeholders below are therefore the people NetLift is designed for, represented by personas, while the DEPI instructor and evaluators are

the only stakeholders who act on the project today. Capabilities under “How NetLift meets them” are planned unless stated otherwise; the task numbers and due dates come from `docs/project-plan.md` (revised 13 October 2026), and a capability counts as implemented only once its pull request is merged. **Influence** is the team’s own assessment of how much a stakeholder decides whether the system is used or accepted; for the business roles it is their influence in a real deployment.

Stakeholders

Stakeholder	Role	Interest in NetLift	Influence	Needs	How NetLift meets them	What we must not claim to them
Marketing analyst (retention / CRM)	Selects a customer base, sets a budget and a per-contact cost, and hands a target list to campaign operations.	A target list that spends the budget on customers whose behaviour changes, not on customers who would have converted anyway.	Medium	A ranked list with an expected incremental-conversion figure and an uncertainty range; a clear contact / suppress / no-action recommendation; a clear “we do not know” state.	Planned: the scoring response (`uplift_score`, `uplift_ci_95`, `recommendation` and a separate `verdict`) is frozen in T36 (due 20 Oct); the FastAPI service is T59 (due 20 Nov) and the planner tab is part of the dashboard, T62 (due 27 Nov). The two abstention verdicts (`insufficient_evidence`, `insufficient_control_data`) are to be rendered visually distinct from a confident result, as the UI/UX guidelines (T49, due 4 Nov) will require.	That the figure is realised money (it is estimated at stated cost and margin on held-out experimental data); that the replay is a live campaign; that the effect applies to customers who actually saw the offer (the MVP estimates the effect of being selected, ITT); any accuracy or F1 figure.
Marketing manager	Decides whether to commit spend, comparing candidate policies.	Evidence that the uplift policy is worth running instead of the response-model policy, at the same budget.	High	One screen comparing uplift, response model, random and blanket contact at equal budget, with intervals; the break-even ratio (cost / margin) exposed as an input.	Planned: the Policy Comparison tab (T62, due 27 Nov), fed by the campaign policy simulator (T61, due 20 Nov), with K1 as the central comparison (T44, due 6 Nov); the cost and margin assumptions are to appear on the same line as the result.	That uplift beats the response model before K1 is measured: an interval that contains zero is a reportable finding, not a failure; that a customer on the suppression list is a “sleeping dog” (we estimate a confidently negative effect for a subgroup; no dataset labels individuals).

Stakeholder	Role	Interest in NetLift	Influence	Needs	How NetLift meets them	What we must not claim to them
Data scientist	Consumes the scoring API, inspects model evidence, retrains.	Reproducible, inspectable results and a fair model comparison.	Medium	Single and batch scoring with the model version recorded on every score; Qini curves and metrics with bootstrap intervals; tracked, reproducible runs; tests that fail when a forbidden column reaches the model matrix.	Planned: `model_version` in every scoring response (schema T36, due 20 Oct; FastAPI service T59, due 20 Nov); an evaluation harness with Qini, AUUC, uplift@k and bootstrap intervals within arms (T39, due 30 Oct); MLflow tracking that records the commit, dataset hash and seed (T41, due 30 Oct); `validate_features()` rejecting forbidden columns by name, with a unit test asserting that it fails (pipeline T37, due 27 Oct; CI enforcement T58, due 20 Nov); a one-command reproduction of the headline result for K8 (T67, due 30 Nov).	That any model is "the best" without a measured interval on the locked test split; that the neural estimator was favoured (it is a challenger, and the tree baseline may win); that `segment_label` is ground truth (it is derived); that results hold on non-experimental data; that the test split was used for tuning.
Campaign operations	Executes the contact list through a campaign tool and keeps the records.	A list they can act on without interpreting model output, and a record of who was contacted and who was held back.	Medium	An unambiguous action per customer; an explicit difference between "do not contact" and "we do not know"; assignment recorded at the time it is made.	Planned: `recommendation` and `verdict` are separate fields in the frozen scoring response (T36), and the dashboard (T62) is to render an empty recommendation differently from a confident one. The Campaign Execution System is drawn in the use case diagram (T46) as an external, deployment-only actor.	That NetLift sends or executes campaigns; that `no_action` or an abstention verdict means "suppress"; that treatment assignment is being recorded for a live campaign.
Finance / audit	Reads the incremental-profit report and the evidence behind it.	A number they can audit, with the assumptions and the control group that produced it.	High	Incremental profit per unit of spend with an interval and the cost and margin named beside it; the number of control customers and control conversions behind each figure; the same figure reproducible from the repository.	Planned: replay output with incremental profit per unit of spend (K3, simulator T61) reported with intervals and stated assumptions, and a one-command reproduction of the headline result (T67). We also flag how few control conversions sit behind each decile: about 406 on average if they spread evenly. That follows from the counted totals in the specification (2,096,937 control rows, 4,063 control conversions at 0.1938%, so about 210,000 control rows per decile). The 406 is an even-spread average, not a per-decile count, and the deciles that matter most will hold fewer; this is why intervals are mandatory.	"Saved the company $X" (write "estimated a $X incremental-profit opportunity at the stated contact cost and margin, measured on held-out experimental data"); a point estimate without its interval; that profit is realised rather than estimated.
Customers whose data is scored	The people behind the rows; they are never contacted by this project.	That they are not harmed by being scored, profiled or treated differently.	Low (highest stake)	No personal data processed; no manipulative or vulnerable-customer targeting; no unexamined discrimination	Both datasets are public and de-identified, and `.gitignore` excludes raw data (`data/raw/*`, keeping only `data/raw/.gitkeep`), so the	That real individuals were scored or contacted; that the system has been checked for fairness beyond what anonymised Criteo features

Stakeholder	Role	Interest in NetLift	Influence	Needs	How NetLift meets them	What we must not claim to them
				between segments; abstention instead of false confidence.	repository holds loader scripts and checksums rather than data. The scope of acceptable use is stated in the final report (T69, Mahmoud); the targeting-rate check across Hillstrom segments is a table in the error analysis (T65, Ali), and it is structural only on Criteo. The control-group holdout applies to a real deployment: in the replay nobody is withheld, because the control group is already in the data.	allow (the segment check is structural only, and the report says so); that the method is safe for targeting vulnerable customers.
DEPI instructor and evaluators	Assess the project against the DEPI documentation guidelines and the final evaluation.	A complete, honest project in which each member's contribution can be shown.	High	Every required artefact (use cases, requirements, architecture, ERD, DFD, sequence / activity / state / class diagrams, wireframes, UI/UX guidelines, API documentation, test plan); evidence of what each member built; scope and limitations stated rather than hidden.	The documentation set built across the milestones; the GitHub record (issue assignees, pull-request authors, reviewers) as evidence of individual work; computer vision and NLP stated as absent, with the instructor's review recording that they are not required.	That the replay is a live campaign; that anything in the deployment-only list was built or exercised; that any metric is valid without an interval; that CV or NLP are covered; that a figure was verified against the downloaded files when it was taken from documentation.

Needs met this semester, and needs that belong to a real deployment

"This semester" means the need is in scope for the deliverables and is met once the matching deliverable passes review.

Need	This semester	Deployment only	Why
Ranked target list with uplift interval, for a given budget, cost and margin	yes	no	Planned: scoring API (T59) and planner tab (T62) run on the replay scenario over the held-out split.
Compare uplift, response model, random and blanket at equal budget, with intervals	yes	no	This is K1 (T44) and the replay simulator (T61).
Abstention verdicts distinct from a confident recommendation	yes	no	A functional and non-functional requirement; no recommendation is issued without a verdict field.
Model version recorded on every score; reproducible headline result	yes	no	`model_version` is in the output schema (T36, due 20 Oct); MLflow tracking is T41 (due 30 Oct); the K8 reproducibility check is T67.
Incremental result measured against a control group, with its size reported	yes	no	The replay uses the control arm of the recorded experiment.
Permanently maintained randomized holdout for live campaigns	no	yes	In production the counterfactual has to be manufactured by withholding contact from a randomized share; it is designed for, not exercised.
Realised (not estimated) incremental profit	no	yes	Needs a real outcome window and real spend; this semester only produces an estimate under assumed cost and margin.

Need	This semester	Deployment only	Why
Integration with a campaign execution system; assignment recorded at assignment time	no	yes	The Campaign Execution System is an external, deployment-only actor.
Retraining run	yes	no	An offline pipeline job, in scope this semester (ADR 0002, PR #95).
Scheduled retraining trigger (drift)	no	yes	Deployment-only (ADR 0002, PR #95).
No personal data processed	yes	no	Both datasets are public and de-identified.
Handling real customers' personal data (consent, access control, retention)	no	yes	Not exercised; the project processes no personal data.
Scope of acceptable use	yes	no	Stated in the final report (T69).
Segment targeting-rate check	yes	no	A table in the error analysis (T65); on Criteo it is structural only because the features are anonymised.
Required DEPI artefacts and per-member evidence of contribution	yes	no	Produced across the milestones; the GitHub record is the evidence.

Sources

Evidence: `docs/project-plan.md` (task numbers and due dates), the T16 dataset figures verification (Criteo rates and row counts), and the specification, sections: Target users, Business value, Meeting the DEPI Documentation Requirements, Ethical and Privacy Considerations, What a Real Deployment Would Additionally Require, Model Output Schema, Campaign Policy Simulator (wording rules).

2. User Stories and Use Cases

Author: Ali Alaa · Task: T32 · Reviewer: Aliaa Gomaa Date: 13 October 2026 (first version) · Revised: 16 October 2026 (after the second review of PR #93 and the review of ADR 0002, PR #95) Issue: #44 · Pull request: #93 · Milestone: M3 – Requirements Gathering

Section numbers (§) refer to `NetLift_Project_Specification.pdf` (55 pages, snapshot of 13 October 2026, pull request #27 awaiting final review). Every figure, field name, endpoint and rule below is taken from that specification. Where the specification is silent or two statements disagree, the point is marked **[Q#]** and listed in Section 5. Where this document **proposes** a rule the specification does not state, it is labelled **Proposed** and tied to a question, so that nobody mistakes a proposal for a decision. The Markdown copy of the specification is unnumbered and cross-references the wrong numbers, so the PDF numbers are used **[Q8]**. ADR 0002 (PR #95) fixes how the recorded replay (UC-01, UC-06), UC-07 and UC-10 run, and keeps the API at the six endpoints of §35; this document follows it **[Q3]**.

Status legend used throughout:

Status	Meaning
Yes	Built and exercised this semester
Partly	Part of it is built this semester; the rest is designed or documented only
Deployment-only	Not built and not claimed. Recorded because a live system would need it (§15)

Glossary

Short definitions for the readers of T33 and T34 and for Finance. Full definitions are in the specification and in `docs/glossary.md`.

Term	Meaning in NetLift
Uplift / CATE	The estimated change in a customer's outcome caused by contact, $\tau(x) = E[Y(1) - Y(0) \mid X = x]$. Never observed for any single row (§2.2).
ITT (intention-to-treat)	The effect of *being assigned* to the campaign, whether or not the contact was received. The only estimand the decision layer and the simulator use (§6, §10).
Wald (ToT / LATE)	The effect among *compliers* (users actually exposed), computed as ITT divided by P(exposed | treated). Criteo only; about 27.7 × ITT. Reported beside ITT for interpretation and **never** used in $m \cdot \hat{\tau} - c$ (§6, §10).
Qini coefficient	The area between the model's Qini curve and the random-targeting diagonal, normalised by the area of the optimum curve (§27.2).
AUUC	Area under the uplift curve. Normalised differently from Qini, so the library and settings are always named (§27.2).
uplift@k	Uplift measured on the top-k fraction, i.e. what a fixed budget buys (§27.2).
Bootstrap interval	95% percentile interval from at least 1,000 resamples drawn within each treatment arm (§28).

Term	Meaning in NetLift
SMD	Standardized mean difference between treated and control for one covariate; the randomization-integrity check flags |SMD| > 0.1 (§20); K6 passes only when every |SMD| < 0.1.
PSI / KS	Population Stability Index and Kolmogorov–Smirnov statistic: feature-drift measures against the training reference (§32).
c, m, k, c/m	Cost per contact, margin per conversion, number of contacts k = budget / c, and the break-even ratio c/m (§10).
Policy mode vs. policies compared	Policy mode (Fixed budget / Profit-maximising / Fixed target / Suppression-only) is different from the policies compared (Uplift / Response model / Blanket / Random). The policy mode is how the Uplift Policy selects customers; the policies compared are the four systems compared in UC-06 and UC-07 (§10, §13).
Verdict	One of `confident`, `insufficient_evidence`, `insufficient_control_data` (§11, §12). Separate from `recommendation` on purpose.
Replay	A simulated campaign over a held-out slice of historical experimental data: decide without reading outcomes, then reveal and measure (§14). Not a live campaign.
Holdout	In the replay: the selected customers whose *recorded* assignment was control **[Q10]**. In deployment: a permanent randomized share withheld before contact (§15).
Control rate	In the replay: the outcome rate among the holdout. The treated rate minus the control rate is the measured incremental effect (§14) **[Q10]**.
K1	Incremental conversions versus the response model at equal budget (primary KPI). Success: the 95% interval of the difference excludes zero in favour of uplift.
K2	Qini coefficient and AUUC. Success: interval entirely above zero (per `config/kpis.yaml`; §46 says “excludes zero”, which is looser) **[Q11]**.
K3	Incremental profit per unit of spend, with cost and margin stated on the same line.
K4	Negative control: full pipeline on a permuted treatment indicator; pass if the 95% interval of the uplift contains zero.
K5	Calibration: predicted versus measured incremental conversions per decile.
K6	Randomization integrity: every |SMD| < 0.1 and treatment-predictability AUC near 0.5.
K7	Leakage control: a forbidden column reaching the model matrix fails the build.
K8	Reproducibility: clean checkout plus one command reproduces the recorded headline number.
K9 to K11	Serving performance, engineering quality (CI green on every merge), documentation completeness.

Use-case index

The ID list below is the one posted to the team WhatsApp group (due Wednesday 14 October, 22:00). IDs are **frozen**: nothing in this revision renumbers or renames a use case or a story.

ID	Use case	Primary actor	Implemented this semester
UC-01	Define a replay scenario	Marketing Analyst	Yes (scenario configuration read by the offline replay job, ADR 0002)

ID	Use case	Primary actor	Implemented this semester
UC-02	Score a batch of customers	Marketing Analyst	Yes for batch and single scoring. The scheduled trigger is deployment-only **[Q7]**
UC-03	Review model evidence	Data Scientist	Yes
UC-04	Deliver the target list to a campaign	Marketing Analyst	Partly (the download is built; delivery to a Campaign Execution System is deployment-only)
UC-05	Allocate under budget	Marketing Analyst	Yes
UC-06	Reveal outcomes and measure	Marketing Analyst	Yes (offline command-line job, ADR 0002)
UC-07	Compare policies	Marketing Manager	Yes (re-runs the replay through `POST /policy/simulate`, ADR 0002)
UC-08	Inspect a customer explanation	Marketing Analyst	Yes
UC-09	Review drift	Data Scientist	Partly (feature and score drift built; effect drift is mocked)
UC-10	Retrain the model	Data Scientist	Partly **[Q9]**: an offline pipeline job (ADR 0002). The scheduled trigger is deployment-only **[Q7]**

The eight use cases named in the specification (§45) are UC-01, UC-02, UC-05, UC-06, UC-07, UC-08, UC-09 and UC-10. UC-03 and UC-04 are added because the dashboard (§40, Tab 3) and the Campaign Execution System actor (§45) need a use case each.

Typical order in the replay lifecycle (§14): UC-01 → UC-02 → UC-05 → UC-06 (reveal and measure) are steps of one offline replay job in a recorded run (ADR 0002, decision 1). UC-04 (download), UC-07 and UC-08 follow the run. UC-03, UC-09 and UC-10 run alongside it. The numbering of UC-04 before UC-05 is historical; the order above is the one that matters.

---

1. Actors

Actor	Type	Role in NetLift	Source
Marketing Analyst	Primary, human	Defines scenarios, sets cost, margin, budget and policy mode, scores customer lists, downloads the target list, inspects individual customers. Called “Retention / CRM analyst” in the target-user table.	§1.5 target users; §45
Marketing Manager	Primary, human	Compares candidate policies on one screen before committing spend.	§1.5 target users
Data Scientist	Primary, human	Consumes the scoring API, inspects Qini curves and drift monitors, starts retraining runs. Called “Data science team” in the target-user table.	§1.5 target users; §45
Finance / Campaign Audit	Secondary, human (reader)	Reads the incremental-profit report and the size of the holdout (the recorded control arm) that produced it. Never changes anything.	§1.5 target users
Scheduler	Supporting, internal system. **Deployment-only** (working assumption, to be confirmed **[Q7]**)	Would start scheduled batch scoring, scheduled drift reports and the scheduled part of the retraining rule. Not built this semester.	§45; §35 (batch path); §32

Actor	Type	Role in NetLift	Source
Campaign Execution System	**External system, deployment-only**	Would receive the approved target list, run the real campaign and return real outcomes after an outcome window. Does not exist in this project.	§45; §15

Notes on the actor list:

- The specification names two overlapping lists. The use-case-diagram list (§45) has four actors: Marketing Analyst, Data Scientist, Scheduler, Campaign Execution System. The target-user table (§1.5) has four users: CRM analyst, marketing manager, data science team, finance / audit. This document keeps all six names so that no stakeholder disappears. T31 (stakeholders) and T46 (use-case diagram) must use the same names **[Q6]**.
- The MLflow Model Registry, the PostgreSQL scoring log and the FastAPI service are components, not actors. They appear in preconditions and flows only.
- The Scheduler and the Campaign Execution System are both drawn as deployment-only on the use-case diagram. The Scheduler appears as a secondary actor in UC-02, UC-09 and UC-10 and is never required for a flow to work this semester. The Campaign Execution System appears in UC-04 only.

---

2. User stories

Each story has acceptance criteria in Given / When / Then form. “Verdict” always means one of `confident`, `insufficient_evidence`, `insufficient_control_data` (§11, §12).

ID	Role	Capability (short)	Use cases
US-01	Marketing Analyst	Define a replay scenario	UC-01
US-02	Marketing Analyst	Score customers, as a batch or one at a time	UC-02
US-03	Marketing Analyst	Get “insufficient evidence” instead of a guess	UC-02, UC-05, UC-08
US-04	Marketing Analyst	Allocate a budget and download the target list	UC-04, UC-05
US-05	Marketing Analyst	Reveal outcomes only after the decision is fixed, and measure	UC-06
US-06	Marketing Manager	Compare four policies at an identical budget	UC-07 (needs a measured scenario from UC-06)
US-07	Finance / Campaign Audit	Audit every profit figure and its assumptions	UC-06, UC-07
US-08	Marketing Analyst	Inspect one customer’s explanation	UC-08
US-09	Data Scientist	See the model evidence and the drift monitors	UC-03, UC-09
US-10	Data Scientist	Retrain under a documented rule and register the result	UC-10

US-01 Define a replay scenario

As a Marketing Analyst, **I want** to define a replay scenario with a per-contact cost, a margin per conversion, a budget and a slice of held-out data, **so that** the system can replay a campaign decision on data whose outcomes it has not read yet.

Acceptance criteria:

1. **Given** a champion model is registered and a held-out slice exists, **when** I submit cost, margin, budget and a slice, **then** a scenario is stored in state `defined` and the break-even ratio c/m is shown to me.
2. **Given** any of the four inputs is missing (cost, margin and budget must be numbers greater than zero, because k = budget / c and c/m need them; the slice must be selected), **when** I submit, **then** no scenario is created and the missing or invalid input is named.
3. **Given** a scenario in state `defined`, **when** any step before the reveal runs, **then** the data frame it receives contains no outcome column, and an automated test asserts this (§14).

US-02 Score customers

As a Marketing Analyst, **I want** to upload a customer file, or send a single customer, and receive an uplift estimate with a 95% interval and a verdict for each, **so that** I can decide whom to contact.

Acceptance criteria:

1. **Given** a CSV file or JSON array containing only permitted feature columns, **when** I request batch scoring (`POST /score/batch`), **then** I receive a scored file and a summary, and every row carries all eleven fields of the model output schema (§12).
2. **Given** one customer, **when** I call `POST /score`, **then** I receive the full response object, including `model_version`.
3. **Given** the input contains a forbidden column (a treatment indicator, an outcome, or Criteo's `exposure`), **when** I request scoring, **then** the run is rejected, the column is named, and no row is scored (§18, K7).
4. **Given** malformed input or a failed data-quality check (null rates, range violations, schema change), **when** I request scoring, **then** the run is blocked with an error and no score is presented as valid (§32).

US-03 Abstention instead of a guess

As a Marketing Analyst, **I want** the system to say it does not know when the evidence is too thin, **so that** I never act on noise.

Acceptance criteria:

1. **Given** a customer or segment whose 95% bootstrap interval straddles zero, **when** it is scored, **then** the verdict is `insufficient_evidence` and no directional recommendation is issued (§11).
2. **Given** a segment with fewer than the configured minimum number of control-group outcome events, **when** it is scored, **then** the verdict is `insufficient_control_data` and the threshold value used is logged with the response (§11).
3. **Given** either of these verdicts, **when** the dashboard shows the result, **then** it is rendered in a style visually distinct from a confident verdict (§11, §40).
4. **Given** any score produced by the system, **when** the system returns it, **then** it carries a verdict field, because no recommendation is issued without one (§45, non-functional requirements).

US-04 Allocate under budget and download the list

As a Marketing Analyst, **I want** to pick a policy mode and a budget and receive the list of customers to contact, **so that** spend goes to customers whose outcome is expected to change.

Acceptance criteria:

1. **Given** scored customers, per-contact cost c, margin m and budget B, **when** I run the allocation with *Fixed budget*, **then** the customers with verdict `confident` are ranked by estimated uplift and the top k = B / c are selected, and any customer with estimated uplift ≤ 0 is not selected even if budget is left unspent (§10) **[Q2]**.
2. **Given** the same inputs, **when** I run the allocation with *Profit-maximising*, **then** every eligible customer with estimated uplift > c/m is selected, and the budget is an output, not an input (§10).
3. **Given** a target N of incremental conversions, **when** I run the allocation with *Fixed target*, **then** the smallest set expected to deliver N is selected, and the budget required and its uncertainty are reported (§10). What happens when N cannot be reached is open **[Q5]**.
4. **Given** the same inputs, **when** I run the allocation with *Suppression-only*, **then** everyone except negative-uplift customers is selected (§10). Which test defines "negative-uplift" is open **[Q2]**.

Note on criteria 1-4: in a recorded run, the policy mode is set in the scenario config (UC-01); the analyst does not choose it interactively. The "when I run the allocation with …" wording describes the behavior under each mode. 5. **Given** a customer whose estimated uplift is negative and whose 95% confidence interval lies entirely below zero, **when** the allocation runs in any policy mode, **then** that customer is on the suppression list and is never contacted by the policy, regardless of remaining budget (§10) **[Q2]**. 6. **Given** a completed allocation, **when** I open the result in the Campaign Planner, **then** I see the number of customers to contact, expected incremental conversions with a confidence interval, expected incremental profit, and a downloadable target list (§40, Tab 1).

Note on criteria 1 and 5 **[Q2]**: they describe two different mechanisms, not a contradiction. Criterion 1 is a *selection floor* on the point estimate: a customer with an estimate ≤ 0 is simply not selected. Criterion 5 is the *suppression list*, a formal list that needs evidence: the interval must lie entirely below zero. A customer with a negative point estimate whose interval *straddles* zero is therefore not selected, carries the verdict `insufficient_evidence`, and is **not** on the suppression list. Whether the specification intends exactly this reading is the subject of Q2 (b).

US-05 Reveal outcomes and measure

As a Marketing Analyst, **I want** the recorded outcomes to be revealed only after the decision is fixed, and the measured effect compared with the predicted one, **so that** I know whether the decision would have paid off.

Acceptance criteria:

1. **Given** a replay job run whose scenario has reached state `allocated`, **when** the job reaches the reveal step, **then** the state becomes `revealed`, and after measurement `measured` (§14).

2. **Given** a scenario that is not in state `allocated` (not yet allocated, or already revealed), **when** the reveal is requested, **then** it is refused, so each scenario is revealed once.
3. **Given** a revealed scenario, **when** the measurement runs, **then** the measured incremental effect is the treated rate minus the control rate, where the holdout is the selected customers whose recorded assignment was control **[Q10]**, with a bootstrap confidence interval that resamples within treatment arms (§14, §28).
4. **Given** a measured scenario, **when** I open its result, **then** predicted and measured incremental conversions are shown side by side, each with an interval, for the uplift policy, the response-model policy, blanket contact and random contact at an identical budget (§14).
5. **Given** the measurement of a scenario completes, **when** the run finishes, **then** the whole run is logged to MLflow (§14).

US-06 Compare policies

As a Marketing Manager, **I want** to compare uplift targeting, the response model, blanket contact and random contact at an identical budget on one screen, **so that** I can decide which policy to commit spend to.

Acceptance criteria:

1. **Given** a measured scenario, **when** I open *Policy Comparison*, **then** a table shows, for each of the four policies: number contacted, total cost, incremental conversions with a confidence interval, incremental profit, and profit per unit spent (§13), obtained by re-running the replay via `POST /policy/simulate` (UC-07).
2. **Given** the *Policy Comparison* screen is open, **when** I move the budget slider, **then** overlaid cumulative-incremental curves with bootstrap bands are shown and one vertical line moves across all curves at once (§40, Tab 2).
3. **Given** the 95% bootstrap interval of the difference between the uplift policy and the response model excludes zero in favour of uplift, **when** the comparison is shown, **then** the screen states the comparison in words (K1).
4. **Given** the 95% bootstrap interval of that difference contains zero, **when** the comparison is shown, **then** the screen reports that the policies are not distinguishable on this data and does not name a winner (§28, K1).

US-07 Audit a profit figure

As a member of Finance / Campaign Audit, **I want** every profit figure to state its cost and margin assumptions, its interval and the size of the holdout (the recorded control arm) behind it, **so that** I can audit it and know it is an estimate, not realised money.

Acceptance criteria:

1. **Given** any incremental-profit figure, **when** I read it on the dashboard or in a report, **then** it is labelled as an estimate, the per-contact cost and margin are named on the same line, and a 95% interval is given (K3, §13).
2. **Given** a measured scenario, **when** I open its result, **then** the size of the holdout that produced the measured effect is shown (§1.5 target users).

3. **Given** any text shown on the dashboard or written in a report, **when** it states a result, **then** it follows the wording rule of §13 (“estimated … at the stated contact cost and margin, measured on held-out experimental data”) and never claims money saved.

US-08 Inspect a customer explanation

As a Marketing Analyst, **I want** to open one customer and see the score, its interval, the segment label, the recommendation, the verdict and the main drivers, **so that** I can explain a decision to a colleague.

Acceptance criteria:

1. **Given** a scored customer, **when** I select the row, **then** the detail view shows the full response object of §12.
2. **Given** the detail view is open, **when** I read the segment label, **then** it is one of `persuadable`, `sure_thing`, `lost_cause`, `sleeping_dog`, `unknown`, and it is presented as an inference, never as ground truth (§12).
3. **Given** the detail view shows the top drivers, **when** I read their description, **then** they are described as features that drive the model’s uplift prediction, never as causes of the customer being persuadable (§31).
4. **Given** a customer whose verdict is `insufficient_evidence` or `insufficient_control_data`, **when** I open the detail view, **then** the verdict is rendered in the visually distinct abstention style (§40).

US-09 See the model evidence and the drift monitors

As a Data Scientist, **I want** to see the evidence behind the registered model and the drift monitors, **so that** I know whether I can trust it, now and over time.

Acceptance criteria:

1. **Given** a registered champion, **when** I open *Model Evidence*, **then** I see the Qini curve with a confidence band, AUUC and Qini coefficient with intervals, the decile table, the uplift calibration plot, the SHAP summary, the negative-control result and the champion–challenger table (§40, Tab 3).
2. **Given** a registered champion, **when** I call `GET /model/info`, **then** it returns the registered version, the training date, and metrics with confidence intervals (§35).
3. **Given** a drift report exists, **when** I open *Monitoring*, **then** I see feature drift (PSI or KS per feature against the training reference), score drift and data-quality status, with alert thresholds taken from configuration (§32).
4. **Given** *Monitoring* is open, **when** I look at the effect-drift panel, **then** it is labelled as mocked from replay output (§40, Tab 4).

US-10 Retrain under a documented rule

As a Data Scientist, **I want** retraining to follow a documented rule and every retrain to be registered as a new model version with its evaluation attached, **so that** the model behind the API is always identifiable and can be rolled back.

Acceptance criteria:

1. **Given** the documented rule (retrain when the scheduled cadence is reached **or** a drift threshold is breached; either one is enough), **when** either condition is met, **then** a retraining run is started: by the Scheduler in deployment, by the Data Scientist in this project (§32) **[Q7] [Q9]**. The OR reading is **Proposed**: an AND would make the cadence pointless.
2. **Given** a completed retrain, **when** it is logged, **then** it is registered as a new model version with its evaluation attached, and the previous version remains available, so a rollback is a version pin (§32, §45 deployment strategy).
3. **Given** a retraining candidate whose model matrix contains a forbidden column, **when** the pipeline runs, **then** the pipeline fails (K7).
4. **Given** a retraining candidate, **when** its negative control is evaluated, **then** a candidate whose 95% interval of the measured uplift does not contain zero does not pass K4.

---

3. Use cases

Every use case has the same fields. "Abstention verdicts" is always stated in the alternative flows: either the full handling of `insufficient_evidence` and `insufficient_control_data`, or a one-line reason why they cannot arise in that use case.

UC-01 Define a replay scenario

- **Primary actor:** Marketing Analyst
- **Secondary actors:** none
- **Interface:** Scenario configuration file, read by the offline replay job (UC-06). No API endpoint (ADR 0002). Stored in `replay_scenario` (§45, ER diagram).
- **Preconditions:**
  1. A champion model is registered in the MLflow Model Registry.
  2. A held-out slice of data exists that can stand in for "next month's campaign" (§14). Which split is allowed at which stage is open **[Q1]**.
  3. The analyst can run the replay job from the command line **[Q3]**.
  4. The analyst sets the policy mode in the config. If not set, the default is Fixed budget. If the mode is Fixed target, N is required **[Q12]**.
- **Main flow:**
  1. The analyst writes the scenario configuration: the dataset and the slice of the held-out split that stands in for the audience.
  2. The configuration also holds the cost per contact c, the margin per conversion m, the budget, and the policy mode (Fixed budget / Profit-maximising / Fixed target / Suppression-only). When the mode is Fixed target, it also holds N (the target number of incremental conversions).
  3. The system validates the four inputs: cost, margin and budget are numbers greater than zero (k = budget / c and c/m need them), and a slice is selected.
  4. The job reports the break-even ratio c/m (a business input; the dashboard exposes it as a slider, §10).

5. The system stores the scenario and the slice membership. The outcome column is withheld from everything that follows until the reveal step of UC-06.
6. The system sets the scenario state to `defined`.

- **Alternative flows:**
  - A1. *Missing or invalid input:* the job rejects the configuration, names the missing or invalid input, and creates no scenario.
  - A2. *An outcome column is found in a frame handed to a step before the reveal:* the run is invalid and the automated test that guards this boundary fails (§14).
  - A3. *Abstention verdicts:* not applicable here. Nothing is scored yet. Both verdicts first arise in UC-02 and are carried into UC-05 and UC-08.
- **Postconditions:** A scenario exists in state `defined`; its inputs (c, m, budget, slice) are recorded; no outcome has been read.
- **Implemented this semester or deployment-only:** Yes.
- **Related user stories:** US-01
- **Specification basis:** §14 (replay loop, step 1); §10 (c/m slider); §45 (state diagram, ER).

UC-02 Score a batch of customers

- **Primary actor:** Marketing Analyst
- **Secondary actors:** Data Scientist (calls the API directly); Scheduler (scheduled runs, **deployment-only [Q7]**)
- **Interface:** `POST /score/batch` (CSV or JSON array → scored file + summary); single-customer variant `POST /score` (→ full response object); Dashboard, Tab 1.
- **Preconditions:**
  1. A champion model is registered and loaded by version at start-up from the MLflow Model Registry (§35).
  2. The preprocessing artefacts, fitted on the training fold only, are available (§23).
  3. The input contains permitted feature columns only (no treatment indicator, no outcome, no `exposure`).
  4. Where scoring is part of a replay, the scenario is in state `defined`.
- **Main flow:**
  1. The analyst submits a customer file (or one customer).
  2. The system checks data quality: null rates, range violations, schema changes (§32).
  3. The system runs the forbidden-column check (`validate_features()`, §18).
  4. The system applies the stored preprocessing and scores each customer: `uplift_score` with a 95% bootstrap interval `uplift_ci_95`, and `decile` (§11, §12).
  5. The abstention logic assigns a `verdict` to every customer (see A3 and A4).
  6. The system fills the remaining schema fields: `customer_id`, `model_version`, `segment_label`, `expected_incremental_profit` (m · uplift − c), `recommendation`, `top_drivers`, `scored_at` (§12). Where m and c come from outside a scenario is open **[Q4]**.

7. The system returns the scored file and a summary (or the single response object) and writes the scoring run and the scores to the scoring log.
8. If the scoring is part of a replay, the scenario state becomes `scored`.

- **Alternative flows:**
  - A1. *Forbidden column present:* the run is rejected, the column is named, no row is scored (K7, §18).
  - A2. *Data-quality failure or malformed input:* the scoring run is blocked and an error is returned. No partial result is presented as valid.
  - A3. *`insufficient_evidence`:* a customer's or segment's 95% interval straddles zero. The `verdict` is `insufficient_evidence`. The score and the interval are still returned, but no directional recommendation is issued. The system does not round the estimate to a decision (§11). **Proposed:** `recommendation` = `no_action` and `segment_label` = `unknown` **[Q2]**.
  - A4. *`insufficient_control_data`:* the segment holds fewer than the configured minimum number of control-group outcome events. The `verdict` is `insufficient_control_data`. The threshold value used is logged with the response (§11). The same proposal for `recommendation` and `segment_label` applies **[Q2]**.
  - A5. *Single customer:* `POST /score` returns one full response object.
  - A6. *Scheduled run:* **deployment-only.** The Scheduler would start the same flow without an analyst (§35, batch path is the primary production mode). This semester every run is started by a person or a test **[Q7]**.
  - A7. *Model not available:* `GET /health` reports not ready and no score is returned.
- **Postconditions:** Every returned row has all eleven schema fields, including a `verdict` (non-functional requirement, §45). The run is recorded in the scoring log. In a replay the scenario is `scored`.
- **Implemented this semester or deployment-only:** Yes for batch and single scoring. The scheduled trigger (A6) is deployment-only **[Q7]**.
- **Related user stories:** US-02, US-03
- **Specification basis:** §11, §12, §18, §32, §35, §45.

UC-03 Review model evidence

- **Primary actor:** Data Scientist
- **Secondary actors:** Marketing Manager (reads the same screen when asked a question)
- **Interface:** Dashboard, Tab 3 (Model Evidence); `GET /model/info`.
- **Preconditions:**
  1. A champion is registered, with its evaluation attached to the registered version (§32).
  2. The evaluation harness has produced Qini, AUUC and uplift@k with bootstrap intervals, and the negative-control result exists (§27, §28).
- **Main flow:**
  1. The Data Scientist opens Model Evidence.
  2. The system calls `GET /model/info` and shows the registered version, the training date, and the metrics with confidence intervals.

3. The system shows the Qini curve with a confidence band, AUUC and Qini coefficient with intervals, the decile table, the uplift calibration plot, the SHAP summary, the negative-control result and the champion–challenger table (§40).
4. The system names the library and settings used for Qini and AUUC, and the number of bootstrap resamples (§27, K2).

- **Alternative flows:**
  - A1. *Negative control fails:* the 95% interval of the measured uplift on a permuted treatment does not contain zero. The screen shows K4 as **fail** and does not hide it (§28).
  - A2. *Champion and challenger not distinguishable:* the bootstrap distribution of their difference straddles zero. The screen says they are not distinguishable on this data, and does not name a winner (§28).
  - A3. *`insufficient_evidence`:* a decile or segment whose interval straddles zero is shown in the abstention style, not as a positive or negative effect (§11 applies to a decile as well as a customer).
  - A4. *`insufficient_control_data`:* a decile or segment with fewer than the configured minimum control events is shown as such, not as an estimate (§11).
  - A5. *No registered model:* `GET /model/info` returns an error and the tab says so.
- **Postconditions:** Nothing is changed (read-only).
- **Implemented this semester or deployment-only:** Yes.
- **Related user stories:** US-09
- **Specification basis:** §27, §28, §29, §31, §35, §40.

UC-04 Deliver the target list to a campaign

- **Primary actor:** Marketing Analyst
- **Secondary actors:** Campaign Execution System (**external, deployment-only**)
- **Interface:** Dashboard, Tab 1 (downloadable target list). No endpoint exists for the deployment-only steps **[Q3]**.
- **Preconditions:**
  1. The scenario is in state `allocated` (UC-05): a target list and a suppression list exist.
  2. For the deployment-only steps: a Campaign Execution System is connected, and a permanent randomized holdout is maintained (§15).
- **Main flow:**
  1. The analyst downloads the target list. *(Built this semester.)*
  2. *(Deployment-only.)* The system sends the target list to the Campaign Execution System and records who was assigned to which arm at assignment time (`treatment_assignment`, `holdout_membership`).
  3. *(Deployment-only.)* The Campaign Execution System runs the campaign.
  4. *(Deployment-only.)* After the outcome window closes (days to weeks), the Campaign Execution System returns real outcomes, and the system ingests them (`outcome_observation`).

5. *(Deployment-only.)* The measurement job computes the realised effect in aggregate against the permanent holdout.

– In this project steps 2 to 5 are **replaced by UC-06**, which reveals recorded outcomes. This is a replay, not a live campaign, and the project does not claim otherwise (§14, §15).

- **Alternative flows:**
  - A1. *Assigned is not received:* a message is not delivered, or a control customer is reached through another channel. The estimate stays intention-to-treat. Compliance and contamination handling is deployment-only (§6, §15).
  - A2. *Outcome window not yet closed:* the realised effect cannot be computed yet. Only feature drift and score drift are available (§32).
  - A3. `insufficient_evidence` / `insufficient_control_data`: customers carrying either verdict are not eligible in UC-05, so they are not on the contact list. They are reported separately as abstentions, so "do not contact" and "we do not know" are not merged (§12). How they are labelled in the exported file is open **[Q2]**.
- **Postconditions:** *(Built.)* A downloadable target list file exists. *(Deployment-only.)* Assignment and holdout membership are recorded; outcomes are ingested; the scenario would reach `measured` on real data.
- **Implemented this semester or deployment-only: Partly.** The download is built. Everything involving the Campaign Execution System is **deployment-only** and not built.
- **Related user stories:** US-04
- **Specification basis:** §6, §14, §15, §40 (Tab 1), §43 and §44 (campaign lifecycle, export, outcome ingestion), §45.

UC-05 Allocate under budget

- **Primary actor:** Marketing Analyst
- **Secondary actors:** none
- **Interface:** The decision layer (`PolicyEngine`), called by the replay job (UC-06) in a recorded run (ADR 0002). None of the six endpoints of §35 returns an allocation. Whether Tab 1 (Campaign Planner, §40) also calls it for an interactive allocation is not decided in ADR 0002 **[Q3]**.
- **Preconditions:**
  1. The scenario is in state `scored` (UC-02): every customer has an uplift estimate, an interval and a verdict.
  2. Per-contact cost c, margin m and budget B are known (UC-01).
  3. The scenario configuration has a policy_mode set (UC-01).
- **Main flow:**
  1. The replay job reads the policy_mode from the scenario configuration (UC-01) and applies it (§10):
     - *Fixed budget:* contact the top k by estimated uplift, k = B / c; suppress any selected customer whose estimated uplift ≤ 0, even if budget is left unspent.
     - *Profit-maximising:* contact every customer whose estimated uplift > c/m; the budget is an output, not an input.
     - *Fixed target:* contact the smallest set expected to deliver N incremental conversions (N comes from the config, UC-01); report the budget required and the uncertainty on it.

- *Suppression-only:* contact everyone except negative-uplift customers.

2. The decision layer builds the **eligible pool**: the customers whose verdict is `confident`. Customers with `insufficient_evidence` or `insufficient_control_data` are not eligible for a directional recommendation. They are reported separately and are never used to fill k. **Proposed [Q2]:** the estimate of an abstained customer carries no evidence, so spending budget on it is spending on noise, which the abstention path exists to prevent.
3. The decision layer applies the chosen mode to the eligible pool, working on the ITT-scale uplift. The Wald estimate never enters m · uplift − c (§10, §6). Under *Fixed budget* the eligible customers are ranked by estimated uplift and the top k are taken, so k is filled from the next eligible customers in the ranking. If fewer than k eligible customers have an estimated uplift above zero, only those are taken and the remaining budget stays unspent. **Proposed [Q2]:** leaving k short while eligible customers remain would reduce spend without a reason.
4. The decision layer applies the sleeping-dog rule: any customer whose estimated uplift is negative and whose confidence interval lies entirely below zero goes on the suppression list and is never contacted by the policy, regardless of remaining budget (§10).
5. The system records the **replay holdout**. A replay runs on a slice that already contains a randomized control arm, so the holdout is not drawn afresh: after the target list is fixed, the system joins the *recorded* treatment indicator to the selected customers. Those whose recorded assignment was control form the holdout (the not-contacted comparison group); those whose recorded assignment was treated form the contacted group. This is valid because the selection used only pre-treatment features, so within the selected set the recorded assignment is still random (§5). The scoring and allocation functions receive neither the outcome nor the treatment column; the join happens after allocation. The holdout rule (the control arm of the selected targets) is **decided in ADR 0002 [Q10]**; withholding the treatment column from scoring and allocation is **Proposed**. In a live deployment the holdout is instead a permanent randomized share withheld before contact (UC-04, §15). This record is what makes UC-06 possible (§14, step 4).
6. The system shows: the number of customers to contact, expected incremental conversions with a confidence interval, expected incremental profit, and a line comparing the policy with targeting by response score (§40).
7. The scenario state becomes `allocated`.

- **Alternative flows:**
  - A1. *k = 0, or every estimated uplift ≤ 0:* the target list is empty, and under *Fixed budget* the budget stays unspent (§10).
  - A2. *Fixed target N cannot be reached:* behaviour not specified **[Q5]**.
  - A3. *`insufficient_evidence`:* a customer whose interval straddles zero is not in the eligible pool (step 2), is shown in the abstention style, and is listed separately (§11).
  - A4. *`insufficient_control_data`:* a customer in a segment with too few control events is treated the same way, and the configured threshold is logged (§11).
  - A5. *Suppression versus abstention:* a customer with a negative point estimate whose interval straddles zero is not on the suppression list, because the sleeping-dog rule needs the interval entirely below zero. The customer is also not selected, because the point estimate is ≤ 0 and the verdict is an abstention. The *selection floor* and the *suppression list* are two mechanisms. Whether the specification intends this reading is open, as is the test used by *Suppression-only* (**Proposed:** *Suppression-only* excludes the suppression list from the eligible pool of step 2) **[Q2]**.

- A6. *Eligible pool smaller than k:* the policy contacts fewer than k customers. The system reports the number actually contacted and the unspent budget next to every figure, and the baseline policies of UC-06 still use the same k, so that the comparison "at an identical budget" is not misleading **[Q2]**.

- **Postconditions:** The scenario is `allocated`. The target list, the suppression list, the abstention list and the holdout record are stored. Steps 1 to 5 never touched the outcome column, and a test asserts it (§14).
- **Implemented this semester or deployment-only:** Yes.
- **Related user stories:** US-03, US-04
- **Specification basis:** §5, §10, §11, §14, §15, §40.

UC-06 Reveal outcomes and measure

- **Primary actor:** Marketing Analyst
- **Secondary actors:** Finance / Campaign Audit (reads the result)
- **Interface:** Offline command-line job (replay job), logged to MLflow. No API endpoint (ADR 0002). Recorded runs are shown in Tab 2 and Tab 4 (ADR 0002); how the dashboard reads them is for T45 and T62. Who runs the command is open **[Q3]**.
- **Preconditions:**
  1. A valid scenario configuration exists (UC-01) and a champion model is registered.
  2. One run covers §14 steps [1]–[8] (ADR 0002): before the reveal the job runs UC-01, UC-02 and UC-05, and none of them reads the outcome column (§14).
  3. The slice used is one that may be revealed at this stage of the project **[Q1]**.
- **Main flow:**
  1. The analyst starts the replay job from the command line with the scenario configuration.
  2. The job runs UC-01, UC-02 and UC-05 for the scenario (states `defined`, `scored`, `allocated`) without the outcome column, including the holdout record (UC-05 step 5).
  3. The job checks that the scenario is `allocated` and has not been revealed before, then reads the recorded outcomes of the slice for the first time. State: `revealed`.
  4. The job measures, for each policy, the incremental effect as the treated rate minus the control rate (the holdout: the selected customers whose recorded assignment was control **[Q10]**), with a 95% bootstrap interval that resamples within treatment arms (§14, §28, at least 1,000 resamples, K2).
  5. The job replays the four policies once, at an identical budget (the same replay code that UC-07 re-runs): uplift, response model, blanket contact and random contact. The Uplift Policy uses the policy_mode from the config (UC-01). The other three (response model, blanket, random) are fixed, as §10 and §13 define them. For each it produces the number contacted, total cost, predicted and measured incremental conversions each with an interval, incremental profit and profit per unit spent (§13, §14), and the cumulative curves with bootstrap bands.

6. The job converts the result to incremental profit using the stated cost and margin (K3), and records predicted-versus-measured differences per decile (K5).
7. The job stores the complete result with the scenario (tables, curves, intervals, holdout size, library and settings, and the bootstrap seed), logs the whole run to MLflow, and sets the state to `measured`. Finance reads the stored result and the MLflow run. The what-if of UC-07 does not read it: it re-runs the replay code. The seed is stored so that a re-run reproduces these figures; ADR 0002 only requires that the same inputs and seed give the same result.

- **Alternative flows:**
  - A1. *Reveal requested when the state is not `allocated`:* refused, whether the scenario was not allocated yet or was already revealed or measured, so each scenario is revealed once (§14). Measuring again on different inputs needs a new scenario (UC-01) **[Q1]**.
  - A2. *`insufficient_evidence`:* a measured effect whose 95% interval contains zero is reported as not distinguishable from zero and is shown in the abstention style. It is not rounded to a decision (§11, K1).
  - A3. *`insufficient_control_data`:* a policy or decile whose holdout holds fewer than the configured minimum number of control events is reported as `insufficient_control_data`, and no measured effect is claimed for it (§11). Because the replay holdout is only the recorded control share of the selected customers, about 15% on Criteo (§16.1), this is an expected outcome and not a rare one **[Q10]**.
  - A4. *Outcome leak detected:* if the test that asserts "no outcome column before the reveal" fails, the run is invalid and its result is not reported (§14).
- **Postconditions:** The scenario is `measured`. A stored result and a calibration record exist. The run is in MLflow. Every figure is an *estimated* incremental effect on held-out experimental data, not realised money (§13).
- **Implemented this semester or deployment-only:** Yes, as a simulated replay over recorded outcomes, run as an offline job. A live campaign is deployment-only (UC-04, §15).
- **Related user stories:** US-05, US-07
- **Specification basis:** §13, §14, §28, §35, §46 (K1, K3, K5).

UC-07 Compare policies

- **Primary actor:** Marketing Manager
- **Secondary actors:** Marketing Analyst; Finance / Campaign Audit
- **Interface:** Dashboard, Tab 2 (Policy Comparison); `POST /policy/simulate` (re-runs the replay code).
- **Preconditions:**
  1. The scenario is in state `measured` (UC-06): its allocation, holdout record, replay seed and stored result exist. Tying a simulate call to a recorded scenario is **Proposed**: ADR 0002 names only budget, cost and margin as its inputs **[Q3]**.
  2. The champion model and the response-model baseline both took part in that result.
  3. The scenario's slice is not the locked test split: a re-run reads the recorded outcomes again **[Q1]**.

- **Design decision (re-run, not reuse; ADR 0002, decision 2):** UC-07 re-runs the replay code through `POST /policy/simulate` for a given budget, cost and margin. It simulates again; it does not read a stored UC-06 result. Grounds given by the ADR: (1) a simulate call is an interactive what-if and creates no recorded run; (2) the offline job and the endpoint call one replay implementation, and a test asserts that the same inputs and seed give the same result; (3) the API keeps its six endpoints (decision 4), so the existing endpoint serves this screen. Trade-off stated by the ADR: the dashboard cannot start a recorded replay, it shows recorded results, and the interactive part is the simulate call. This document's own note: a re-run reads the recorded outcomes again, so the reveal-once rule is stated precisely in **[Q1]** and the locked test split is excluded (A5). A different slice or a different model is a new scenario (UC-01).
- **Main flow:**
  1. The Marketing Manager opens Policy Comparison (Tab 2, which shows the recorded runs, ADR 0002) and selects a measured scenario.
  2. The system calls `POST /policy/simulate` with the scenario's budget, cost and margin. The endpoint re-runs the replay code with the recorded seed. It uses the policy_mode from the scenario's configuration (UC-01) for the Uplift Policy. It returns the policy comparison table (§35).
  3. The system shows one table: number contacted, total cost, incremental conversions with a confidence interval, incremental profit, and profit per unit spent, for each policy (§13), with the size of the holdout behind each figure (US-07).
  4. The system shows overlaid cumulative-incremental curves with bootstrap bands.
  5. The manager moves the budget slider; one vertical line moves across all returned curves at once without a new call (§40). Changing the budget, cost or margin used for the table is a new call to `POST /policy/simulate`, and the new assumptions are named on the same line as the figure.
  6. The system states the headline comparison in words, with its interval (K1).
- **Alternative flows:**
  - A1. *`insufficient_evidence`:* the 95% interval of the difference between the uplift policy and the response model contains zero. The screen reports that the policies are not distinguishable on this data and names no winner. This is "a reportable finding, not a failed project" (K1, §28). Applying the abstention state to a policy-level figure extends §11 **[Q2]**.
  - A2. *`insufficient_control_data`:* a policy whose holdout has too few control events is shown in the abstention style instead of a number (§11).
  - A3. *Profit shown as money saved:* not allowed. Every profit figure is labelled estimated, with cost and margin on the same line (§13, K3).
  - A4. *Scenario not yet `measured`:* the screen says the comparison does not exist yet and points to UC-06. No re-run is started.
  - A5. *Scenario on the locked test split:* the re-run is refused, and the screen shows only the figures that UC-06 stored. The locked split is read once (T70, §18). **Proposed [Q1].**
- **Postconditions:** Read-only: the scenario state and the stored result of UC-06 are unchanged, and the call creates no recorded run (ADR 0002). The comparison is available for the final presentation, where the simulator output is the artefact to open with (§13).
- **Implemented this semester or deployment-only:** Yes.
- **Related user stories:** US-06, US-07
- **Specification basis:** §13, §14, §28, §40, §46 (K1, K3).

UC-08 Inspect a customer explanation

- **Primary actor:** Marketing Analyst
- **Secondary actors:** Data Scientist
- **Interface:** Dashboard, customer detail view; `POST /score` returns the same object.
- **Preconditions:** The customer has been scored (UC-02) and the scoring run is recorded.
- **Main flow:**
  1. The analyst selects a customer row.
  2. The system shows the full response object (§12): `customer_id`, `model_version`, `uplift_score`, `uplift_ci_95`, `decile`, `segment_label`, `expected_incremental_profit`, `recommendation`, `verdict`, `top_drivers` (feature and signed contribution), `scored_at`.
  3. The system shows `segment_label` as one of `persuadable`, `sure_thing`, `lost_cause`, `sleeping_dog`, `unknown`, and states that it is an inference from the estimated effect plus the predicted baseline conversion probability, never ground truth (§12).
  4. The system describes `top_drivers` as features that drive the model's uplift prediction, never as causes (§31).
- **Alternative flows:**
  - A1. *`insufficient_evidence`:* the customer's interval straddles zero. The view shows the score and the interval, and renders the verdict in the distinct abstention style, with no directional recommendation (§11, §40).
  - A2. *`insufficient_control_data`:* the customer's segment has fewer than the configured minimum control events. The view shows this verdict in the same distinct style, and the threshold used is visible from the log (§11).
  - A3. *Unknown customer:* the customer is not in the scoring run; the view reports it.
  - A4. *Criteo:* `top_drivers` name anonymised features (`f0` to `f11`), so a plain-language description is possible on Hillstrom only (§29, §52).
  - Whether `segment_label` becomes `unknown` under A1 and A2 is open (**Proposed:** yes) **[Q2]**.
- **Postconditions:** Read-only.
- **Implemented this semester or deployment-only:** Yes.
- **Related user stories:** US-03, US-08
- **Specification basis:** §11, §12, §31, §40.

UC-09 Review drift

- **Primary actor:** Data Scientist
- **Secondary actors:** Scheduler (would produce the drift reports on a schedule; **deployment-only [Q7]**)
- **Interface:** Dashboard, Tab 4 (Monitoring); `GET /monitor/drift` (latest drift report).
- **Preconditions:**
  1. A training reference distribution is stored.

2. At least one scored batch exists to compare against it, and a drift report has been produced from it. This semester the report is produced on demand, by comparing a scored batch with the training reference; scheduled generation is deployment-only. **Proposed [Q7].**
3. Alert thresholds are set in configuration (§32).

- **Main flow:**
  1. The Data Scientist opens Monitoring.
  2. The system calls `GET /monitor/drift` and shows the latest drift report.
  3. The system shows feature drift (PSI or KS statistic per feature, scored batch against the training reference), score drift (distribution of predicted uplift over time; a collapsing spread is an early warning) and data-quality status (null rates, range violations, schema changes) (§32).
  4. The system shows the registered model version, its training date, and the results of each simulated campaign replay (§40).
  5. The system shows the effect-drift panel, mocked from replay output and labelled as such.
- **Alternative flows:**
  - A1. *Threshold breached:* the system raises an alert. The retraining rule may then lead the Data Scientist to start UC-10.
  - A2. *Data-quality failure:* the scoring run is blocked (§32).
  - A3. *No drift report yet:* the tab says so. The specification does not define the response of `GET /monitor/drift` in this case.
  - A4. *Abstention verdicts:* the two verdicts are not produced by the drift monitors. They appear only where the effect-drift panel shows a replay's measured effect: an interval containing zero is shown as `insufficient_evidence`, and too few control events as `insufficient_control_data`.
- **Postconditions:** Read-only.
- **Implemented this semester or deployment-only: Partly.** Feature drift, score drift and data-quality status are built and can be run on historical data split by time or on a simulated shifted population. **Effect drift** needs a permanent randomized holdout and a real outcome window and is **deployment-only**: the panel is mocked. The early-proxy outcome and the predicted-versus-measured calibration after an outcome window are designed and documented, not implemented (§32). Scheduled report generation is deployment-only **[Q7]**.
- **Related user stories:** US-09
- **Specification basis:** §15, §32, §35, §40.

UC-10 Retrain the model

- **Primary actor:** Data Scientist
- **Secondary actors:** Scheduler (scheduled cadence; **deployment-only [Q7]**)
- **Interface:** Offline pipeline job (training pipeline) and the MLflow Model Registry. No API endpoint (ADR 0002).
- **Preconditions:**
  1. A documented retraining rule exists: retrain when the scheduled cadence is reached **or** a drift threshold is breached, either one being enough (§32). The OR reading is **Proposed [Q9]**.
  2. The pipeline is reproducible from a configuration file and a git commit hash (§33, K8).

3. Data for the new run is available.

- **Main flow:**
  1. A trigger arises: a drift threshold is breached (UC-09), or the cadence date is reached. This semester the Data Scientist starts the run by hand; the Scheduler starting it automatically is deployment-only **[Q7]**.
  2. The pipeline re-runs: schema validation, leakage audit, joint stratified split, baselines first, uplift estimators, evaluation with bootstrap intervals (§23).
  3. The candidate is compared with the current champion on the same bootstrap resamples (§28).
  4. The candidate is logged to MLflow as a new registered model version with its evaluation attached; the model lifecycle moves `registered` → `staging` → `production` → `archived` (§45).
  5. The API loads the model version selected at start-up, and returns that version in every response (§35).
- **Alternative flows:**
  - A1. *Forbidden column reaches the model matrix:* the pipeline fails (K7).
  - A2. *Negative control fails:* the candidate does not pass K4.
  - A3. *Candidate not distinguishable from the champion:* the difference distribution straddles zero (§28). The rule for promotion is not specified **[Q9]**.
  - A4. *Rollback:* the previous version stays registered, so a rollback is a version pin (§45).
  - A5. *Abstention verdicts:* not produced by retraining itself. The new version's evaluation reports intervals, and any decile whose interval straddles zero or whose control events are below the minimum is reported as `insufficient_evidence` or `insufficient_control_data` (UC-03).
- **Postconditions:** A new registered model version exists with its evaluation attached. The previous version is retained.
- **Implemented this semester or deployment-only: Partly [Q9].** The retraining rule is documented, and a retraining run is the same reproducible pipeline run again, started by the Data Scientist. The automatic production trigger driven by a Scheduler, and retraining on real outcomes, are **deployment-only**.
- **Related user stories:** US-10
- **Specification basis:** §15, §28, §32, §33, §35, §45.

---

4. Traceability

4.1 User story ↔ use case

	UC-01	UC-02	UC-03	UC-04	UC-05	UC-06	UC-07	UC-08	UC-09	UC-10
US-01	•									
US-02		•								
US-03		•			•			•		

	UC-01	UC-02	UC-03	UC-04	UC-05	UC-06	UC-07	UC-08	UC-09	UC-10
US-04				•	•					
US-05						•				
US-06							•			
US-07						•	•			
US-08								•		
US-09			•						•	
US-10										•

Coverage check: every use case is reached by at least one story, and every story reaches at least one use case. US-06 and US-07 (criterion 2) depend on UC-06 having measured the scenario (stored allocation, holdout record, seed and result): this is a precondition of UC-07, not a second use case of the story. UC-07 re-runs the replay and does not read the stored result (ADR 0002; locked split: UC-07 A5).

4.2 Use case ↔ interface ↔ specification

This table is the input for T33 (each functional requirement must be traceable to an API endpoint and a test), T45 (API contract), T46 (use-case diagram) and T51 (test plan).

Use case	Dashboard	API endpoint (§35)	Specification §	Status
UC-01	none: configuration file	none: read by the offline replay job (ADR 0002)	10, 14, 45	Yes
UC-02	Tab 1	`POST /score/batch`, `POST /score`	11, 12, 18, 32, 35	Yes (scheduled trigger deployment-only **[Q7]**)
UC-03	Tab 3	`GET /model/info`	27, 28, 29, 31, 40	Yes
UC-04	Tab 1 (download)	none **[Q3]**	6, 14, 15, 43, 44	Partly
UC-05	Tab 1 (interactive use open **[Q3]**)	none: decision layer called by the replay job (ADR 0002)	5, 10, 11, 14, 15	Yes
UC-06	Tab 2, Tab 4 (recorded runs; ADR 0002)	none: offline command-line job, logged to MLflow (ADR 0002)	13, 14, 28, 35, 46	Yes
UC-07	Tab 2	`POST /policy/simulate` (re-runs the replay code; ADR 0002)	13, 14, 28, 35, 40, 46	Yes
UC-08	Customer detail	`POST /score`	11, 12, 31, 40	Yes
UC-09	Tab 4	`GET /monitor/drift`	15, 32, 40	Partly **[Q7]**
UC-10	none	none: offline pipeline job (ADR 0002)	15, 28, 32, 33, 45	Partly **[Q7]** **[Q9]**

The API stays at the six endpoints of §35 (ADR 0002). Every other trigger is a dashboard action or an offline job.

`GET /health` (liveness and readiness) is used by UC-02 (A7) and is otherwise an operational endpoint, not tied to a use case.

Suggested convention for T33 and T34: every requirement gets an ID (FR-xx, NFR-xx) that names the use case it comes from, and every test name carries the requirement ID, as the milestone plan requires.

---

5. Open questions and assumptions

Raised from writing this document. None is decided here, except Q3 and Q10, which ADR 0002 has decided and which stay listed for traceability; each needs an owner. **Q1 and Q2 need a numbered record in `docs/decisions/` before T33 is frozen** (working agreement, §6), because requirements and the decision layer depend on them. Items labelled **Proposed** in the text are this document's suggested answers, not agreements.

ID	Question	Why it matters	Suggested owner
Q1	The specification says the locked test split is "touched exactly once, at champion confirmation" (§18), and the plan opens it once on 1 December (T70). Yet K1, K2 and K3 are "measured on the locked test split" and first reported at M4 (§46), and §13 and §14 replay on "the held-out test split". A replay run repeatedly for development, or for every demo, would reopen it. Which split does each replay use at each stage? Proposed: development and demo replays use the validation split or a pre-agreed replay slice, and the locked test split is used once, in T70. Each scenario is also revealed once (UC-06 A1). Because UC-07 re-runs the replay (ADR 0002) and so reads the recorded outcomes again, the reveal-once rule is read as: the allocation is fixed before the reveal and never changed after it, and a re-run leaves the scenario state and the stored result unchanged. On the locked split no re-run is allowed (UC-07 A5). ADR 0002 adds that a job needing the locked split opens it through `LockedTestSplit.open(reason)`, once, under the T70 procedure.	Reopening the locked split is the failure the project exists to avoid	Ali with Mahmoud
Q2	The rules linking `verdict`, `recommendation` and `segment_label`, and the treatment of abstained customers by the decision layer, are not specified. (a) What does `recommendation` hold when the verdict is `insufficient_*`? Proposed: `no_action`. (b) *Fixed budget* does not select any estimate ≤ 0, but the sleeping-dog rule needs the interval entirely below zero, and *Suppression-only* says "negative-uplift customers" without saying which test. Proposed: two mechanisms (selection floor on the point estimate; suppression list on the interval), and *Suppression-only* excludes the suppression list from the eligible pool (see (g)). (c) May `segment_label` be `unknown` under abstention? Proposed: yes. (d) Does the abstention state also apply to a policy-level figure (UC-07)? §11 names customer, decile and policy for the interval but states the verdict rule for a segment. (e) How is k filled when abstained customers are left out? Proposed: build the eligible pool of `confident` customers first, then take the top k from it; never fill k with abstained customers; if the pool has fewer than k, leave the budget unspent. (f) How is "identical budget" kept when the uplift policy contacts fewer than k? Proposed: baselines use the same k, and the number actually contacted and the unspent budget are reported next to every figure. Because individual-level effects are small,	These fields are frozen in T36 and drive T33, T55, T44 and the API tests	Ali (T55) with Mahmoud (T36)

ID	Question	Why it matters	Suggested owner
	the eligible pool may be much smaller than k; its size should be measured early. (g) Under *Suppression-only* (§10: "everyone except negative-uplift customers"), are abstained customers contacted? Proposed: no; the eligible pool of UC-05 step 2 applies in every mode. This is stricter than the wording of §10.		
Q3	The six endpoints of §35 do not cover defining a scenario, allocating, revealing outcomes, comparing policies from a stored scenario, or retraining. **Decided in ADR 0002:** the API keeps the six endpoints and gets no new one. A recorded replay (UC-01, UC-06) is an offline command-line job, logged to MLflow, with no endpoint. UC-07 re-runs the replay code through `POST /policy/simulate` (re-run, not reuse). UC-10 is an offline pipeline job. UC-04 delivery stays deployment-only, with the target list downloadable from the dashboard. Still open, and not answered by the ADR: (a) who runs the replay command (the ADR says only that someone runs it; UC-06's primary actor is the Marketing Analyst, and a command-line job suggests the Data Scientist); (b) whether Tab 1 calls the decision layer for an interactive allocation (UC-05); (c) what a simulate call takes besides budget, cost and margin (a scenario, a slice, a model version); (d) how the dashboard reads recorded runs, `GET /model/info` or MLflow directly, which the ADR leaves to T45 and T62.	T45 freezes the API contract; T33 needs a requirement for each use case	Mazen (T45), Mohammed (T62), Ali (T46, actors)
Q4	`expected_incremental_profit` is m · uplift − c, but `POST /score` and `POST /score/batch` take no m and c in §35. Where do they come from outside a scenario: the request, or configuration?	The response schema cannot be filled otherwise	Ali, Mazen
Q5	*Fixed target:* what happens when no set of customers is expected to deliver N incremental conversions?	Decision-layer edge case (T55)	Ali
Q6	The use-case-diagram actors (§45) and the target users (§1.5) differ. This document keeps six actors. T31 and T46 must use the same names.	Three documents would otherwise describe different actors	Mohammed (T31), Ali (T46)
Q7	The Scheduler is an actor in §45, but the specification does not say whether it is built. This document treats it as **deployment-only**, so scheduled batch scoring, scheduled drift reports and the scheduled retraining trigger are deployment-only, and drift reports are produced on demand (UC-09). Please confirm.	Decides what is "Yes" and what is "Deployment-only" in UC-02, UC-09, UC-10	Mazen, Mahmoud
Q8	`01-specification.md` is unnumbered, and its internal cross-references use different numbers from the PDF (for example, it refers to the schema as Section 9 and the deployment gap as Section 12; the PDF numbers them 12 and 15). This document cites the PDF numbers.	A reviewer following the Markdown copy would land on the wrong section	Mahmoud
Q9	UC-10 and US-10: the specification lists "retrain" as a use case and "a documented retraining rule" as SHOULD HAVE, but does not say whether a retraining run is built or only documented, whether the trigger is OR or AND, or what makes a retrained candidate replace the champion. Proposed: OR (cadence or drift threshold, either is enough; an AND would make the cadence pointless, and §15 says retraining must not be purely reactive, which §32 turns into a rule combining a cadence with a drift threshold); this semester a retraining run is a reproducible pipeline run started by the Data Scientist. A new champion would also normally need confirming on the locked split (see Q1).	Decides the status "Partly" and the acceptance criteria of US-10	Mahmoud
Q10	**Decided in ADR 0002** (with the §14 correction in T52, #94): the replay draws no random holdout; the holdout is the selected targets in the original control arm, and the effect is the treated rate minus the control rate within them, with a bootstrap interval resampled within arms. The original question	Feeds US-05 criterion 3, UC-05 step 5 and UC-06 steps 3 and 4	Ali with Mahmoud (T52)

ID	Question	Why it matters	Suggested owner
	follows. §14 step 4 says “a randomized slice of the selected targets is marked as not-contacted”. In a replay over recorded data a fresh randomization is impossible: a customer who was treated has no outcome for “not contacted”. Proposed reading: the holdout is the selected customers whose *recorded* assignment was control, joined after the target list is fixed; the scoring and allocation functions see neither outcome nor treatment. This is valid because selection uses pre-treatment features only. Consequence: the holdout is the control share of the selection, about 15% on Criteo (§16.1), so `insufficient_control_data` is likely. How the measured per-contact effect is scaled to k selected customers is to be defined in T44. The wording of §14 needs aligning in T52.		
Q11	K2 has two success conditions. §46 says “Interval excludes zero, i.e. demonstrably better than random targeting”; `config/kpis.yaml` says the interval is “entirely above zero” and adds that an interval entirely below zero is worse than random, so “excludes zero” alone is not sufficient. Proposed: use `config/kpis.yaml` (entirely above zero), because “excludes zero” would pass a worse-than-random model; align §46 in T52. K1 is not affected: it already says “in favour of the uplift policy”.	Fixes the K2 pass condition in the harness (T39), the test plan (T51) and the Model Evidence tab	Ali with Mahmoud (T52)
Q12	Which policy mode is the default if not set? [Proposed: Fixed budget]. What happens if Fixed target is chosen without N? [Proposed: reject the config]. Is Tab 1’s interactive allocation still needed, or is the policy mode set only in the config? [Q3].	UC-01 precondition 4, UC-05 step 1 and precondition 3, UC-06 step 5 and UC-07 step 2 all read the policy mode from the config	Ali with Mazen (T45) and Mohammed (T62)

3. Functional Requirements

Task: T33 · **Issue:** #45 · **Milestone:** M3 – Requirements Gathering
Source: `docs/requirements/02-user-stories-and-use-cases.md` (T32, revised 16 October 2026), ADR 0002 (PR #95), and the NetLift Project Specification PDF snapshot dated 13 October 2026.
Status: Draft for review. Requirement IDs are stable within this document; tests listed here are planned verification names, not claims that tests already exist or pass.

1. Purpose and conventions

This document converts the T32 user stories and use cases (UC-01–UC-10) into testable functional requirements. Every use case maps to at least one requirement. Each requirement identifies its source use case, realization path, and planned test name.

- **"Shall"** describes required behavior supported by T32 and ADR 0002.
- **Open [Q#]** means the behavior depends on a question that T32 explicitly leaves unresolved. It must not be treated as a frozen contract until the named decision is recorded.
- **Proposed [Q#]** preserves a proposal from T32; it is not represented as an agreed product rule.
- **Deployment-only** means described for a future live deployment, not implemented or claimed for this semester.
- API names and response fields use the exact spellings in T32 and the specification.

The API contract remains the six endpoints in specification §35: `POST /score`, `POST /score/batch`, `POST /policy/simulate`, `GET /model/info`, `GET /monitor/drift`, and `GET /health`. Per ADR 0002, recorded replay is an offline command-line job and retraining is an offline pipeline job; neither gets a new endpoint. UC-04 campaign-system delivery and automatic scheduled triggers are deployment-only.

2. Functional requirements

2.1 Scenario definition and replay safety (UC-01, UC-06)

ID	Requirement	Source use case	Realized by	Planned verification test	Notes / decision dependency
FR-01	The system shall read a replay scenario from a configuration file and persist the scenario inputs and selected slice membership with state `defined`.	UC-01	Offline replay CLI job; scenario configuration and `replay_scenario` persistence	`test_fr_01_scenario_config_saved_as_defined`	ADR 0002, decision 1. Which data split may be selected at each project stage remains open [Q1].
FR-02	The system shall validate that cost per contact `c`, margin per conversion `m`, and	UC-01	Offline replay CLI job / scenario-config validator	`test_fr_02_invalid_scenario_config_rejected`	Fixed-target mode also requires `N` [Q12]. Default mode and missing-`N` behavior are

ID	Requirement	Source use case	Realized by	Planned verification test	Notes / decision dependency
	budget are numeric and greater than zero, and that a data slice is selected; invalid or missing inputs shall be named and no scenario shall be created.				proposed, not frozen [Q12].
FR-03	The system shall report the break-even ratio `c/m` for a valid scenario.	UC-01	Offline replay CLI job; report output	`test_fr_03_break_even_ratio_reported`	Specification §10.
FR-04	Before the reveal step, the scoring and allocation steps shall not receive or inspect the outcome column; an automated test shall fail if an outcome column reaches a pre-reveal frame.	UC-01, UC-05, UC-06	Replay pipeline boundary; feature/allocation inputs	`test_fr_04_outcome_unavailable_before_reveal`	Required replay safety invariant, specification §14. T32 also proposes withholding treatment from scoring/allocation; keep that separate from outcome-leakage enforcement until confirmed.
FR-05	The system shall start a recorded replay from its scenario configuration, run scenario definition, scoring, allocation, reveal and measurement in order, and log the recorded run to MLflow.	UC-06	Offline command-line replay job using `netlift.utils.tracking`	`test_fr_05_replay_cli_runs_and_logs_to_mlflow`	No replay/reveal API endpoint. T32 leaves the human/operator who starts the command open [Q3(a)].
FR-06	The system shall refuse to reveal outcomes unless the scenario state is `allocated` and the scenario has not already been revealed; a refused reveal shall not produce a measured result.	UC-06	Replay job state-transition guard	`test_fr_06_reveal_requires_allocated_unrevealed_state`	A new scenario is required to measure different inputs [Q1]. UC-07 what-if re-simulation is separately specified and does not mutate the recorded scenario.
FR-07	The system shall retain the scenario's inputs, target allocation, recorded-arm holdout membership, measured result, intervals, bootstrap seed, library and settings, and associate them with the MLflow run.	UC-06	Replay job result persistence and MLflow tracking	`test_fr_07_replay_artifacts_and_seed_logged`	Supports Finance audit and reproducibility (K3, K8).

2.2 Scoring and abstention (UC-02)

ID	Requirement	Source use case	Realized by	Planned verification test	Notes / decision dependency
FR-08	The system shall accept batch-scoring input as CSV or JSON array at `POST /score/batch`, and return a scored file and summary; it shall accept one customer at `POST /score` and return one full response object.	UC-02	`POST /score/batch`; `POST /score`; Dashboard Tab 1 for batch workflow	`test_fr_08_batch_and_single_scoring_contracts`	Specification §35.

ID	Requirement	Source use case	Realized by	Planned verification test	Notes / decision dependency
FR-09	The system shall reject a scoring request containing forbidden columns, including treatment indicator, outcome, or Criteo `exposure`; the error shall name the offending column and no row shall be scored.	UC-02	Scoring input validation / `validate_features()` before either scoring endpoint	`test_fr_09_forbidden_columns_rejected`	Leakage control K7; specification §18.
FR-10	The system shall block scoring when input is malformed or data-quality checks fail, including configured null-rate, range, or schema-change checks, and shall not present partial output as valid.	UC-02	Scoring input validation and data-quality checks before `POST /score` / `POST /score/batch` results are returned	`test_fr_10_scoring_data_quality_failure_blocks_run`	Threshold values come from configuration [T32, UC-09].
FR-11	Every successfully scored row shall contain all eleven output-schema fields: `customer_id`, `model_version`, `uplift_score`, `uplift_ci_95`, `decile`, `segment_label`, `expected_incremental_profit`, `recommendation`, `verdict`, `top_drivers`, and `scored_at`.	UC-02	Both scoring endpoints; response serializer	`test_fr_11_scoring_response_has_all_eleven_fields`	Exact schema from specification §12 and T32.
FR-12	The system shall calculate and return an uplift estimate, its 95% interval, decile, model version, and a verdict for each successfully scored customer.	UC-02	Scoring pipeline; `POST /score` and `POST /score/batch`	`test_fr_12_score_interval_decile_and_verdict_returned`	Every returned score has a `verdict`. Bootstrap interval details follow specification §28.
FR-13	When the 95% interval straddles zero, the system shall assign `insufficient_evidence`, return the score and interval, and issue no directional recommendation.	UC-02	Abstention decision logic in scoring pipeline	`test_fr_13_interval_straddling_zero_abstains`	The specific values for `recommendation` and `segment_label` under abstention are **Proposed [Q2]** (`no_action` and `unknown` respectively), not frozen.
FR-14	When a segment has fewer than the configured minimum number of control-group outcome events, the system shall assign `insufficient_control_data` and log the threshold used with the response.	UC-02	Abstention decision logic and scoring log	`test_fr_14_insufficient_control_events_abstain`	Exact configured minimum is configuration-driven; handling of related fields remains open [Q2].
FR-15	The system shall distinguish both abstention verdicts visually from a confident verdict in the dashboard.	UC-02	Dashboard Tab 1 and customer-detail views	`test_fr_15_abstention_verdicts_visually_distinct`	Specification §11 and §40.
FR-16	If the champion model is unavailable or the service is not ready, the system shall report not-ready through `GET /health` and shall	UC-02	`GET /health`; scoring service readiness guard	`test_fr_16_unready_service_does_not_score`	Operational endpoint under §35; no separate model-scoring endpoint is added.

ID	Requirement	Source use case	Realized by	Planned verification test	Notes / decision dependency
	not return a score as valid.				
FR-17	The system shall populate `expected_incremental_profit` as `m · uplift − c` wherever the required `m` and `c` are available, and shall identify the cost and margin assumptions used for the figure.	UC-02, UC-07	Scoring response and policy-simulation result assembly	`test_fr_17_profit_uses_named_cost_and_margin`	Where `m` and `c` come from for standalone scoring is **Open [Q4]**; do not freeze this contract before resolving request-versus-configuration source.
FR-18	The system shall load the registered champion model version and its fitted preprocessing artifacts for scoring, and return that model version in every scoring response.	UC-02	Scoring service model loader / preprocessing pipeline	`test_fr_18_scoring_uses_registered_version`	Specification §23 and §35.

2.3 Model evidence and explanations (UC-03, UC-08)

ID	Requirement	Source use case	Realized by	Planned verification test	Notes / decision dependency
FR-19	The system shall expose the registered model version, training date, and evaluation metrics with confidence intervals through `GET /model/info`.	UC-03	`GET /model/info`; Dashboard Tab 3	`test_fr_19_model_info_returns_version_date_metrics`	Specification §35.
FR-20	The Model Evidence view shall show the Qini curve and confidence band, AUUC and Qini coefficient with intervals, decile table, uplift-calibration plot, SHAP summary, negative-control result, and champion–challenger comparison.	UC-03	Dashboard Tab 3; evaluation artifacts	`test_fr_20_model_evidence_view_has_required_artifacts`	Name the Qini/AUUC library and settings and bootstrap-resample count in the displayed evidence. K2's success condition is that the interval lies entirely above zero, following `config/kpis.yaml`; specification §46 alignment is tracked by #94, and Q11 remains open until that PR merges.
FR-21	The customer detail view shall show the full eleven-field response object, including the score interval, verdict, segment label, recommendation, model version, and top drivers.	UC-08	Dashboard customer detail; shared response schema from `POST /score`	`test_fr_21_customer_detail_shows_full_response`	Specification §12.
FR-22	The system shall display `segment_label` only from `persuadable`, `sure_thing`, `lost_cause`, `sleeping_dog`, or `unknown`, and describe it as an inference rather than ground truth.	UC-08	Customer detail view and scoring serializer	`test_fr_22_segment_label_values_and_copy`	Whether abstention always maps to `unknown` is **Proposed [Q2]**.
FR-23	The system shall describe `top_drivers` as	UC-08	Customer detail explanation	`test_fr_23_driver_copy_i`	Specification §31 and §29.

ID	Requirement	Source use case	Realized by	Planned verification test	Notes / decision dependency
	features that drive the model's uplift prediction, not as causes of a customer's behavior; on Criteo, the explanation shall preserve anonymized feature labels `f0`–`f11`.		component	`s_non_causal_and_dataset_aware`	
FR-24	The customer detail view shall display either abstention verdict in the distinct abstention style and shall not show a directional recommendation for `insufficient_evidence`.	UC-08	Customer detail view	`test_fr_24_customer_detail_handles_abstention`	If `insufficient_control_data` has a separate recommendation/display rule, resolve with [Q2].

2.4 Target allocation and campaign export (UC-04, UC-05)

ID	Requirement	Source use case	Realized by	Planned verification test	Notes / decision dependency
FR-25	The system shall make the allocated target list downloadable from Dashboard Tab 1.	UC-04, UC-05	Dashboard Tab 1 (Campaign Planner export)	`test_fr_25_allocated_target_list_downloadable`	This is the semester deliverable. Sending targets to a Campaign Execution System, recording live assignment, running a live campaign, ingesting real outcomes and computing live effect are deployment-only; no delivery endpoint is added by ADR 0002.
FR-26	In Fixed budget mode, the policy engine shall rank eligible customers by estimated uplift and select up to $k = B/c$, without selecting customers whose uplift estimate is less than or equal to zero; any unused budget and actual contact count shall be reported.	UC-05	`PolicyEngine`, invoked by the replay job; allocation summary	`test_fr_26_fixed_budget_selection_and_unspent_budget`	Selection floor and abstention interaction are **Proposed [Q2]**. If fewer eligible customers than k remain, the T32 proposal is to leave the remainder unspent [Q2].
FR-27	In Profit-maximising mode, the policy engine shall select every eligible customer whose estimated ITT uplift is greater than c/m; the required spend is an output rather than an input.	UC-05	`PolicyEngine` / replay job	`test_fr_27_profit_maximising_uses_break_even_ratio`	Use ITT-scale uplift, not Wald/ToT/LATE uplift. Eligibility of abstained customers remains subject to [Q2].
FR-28	In Fixed target mode, the policy engine shall select the smallest set expected to deliver target N and report the required budget and its uncertainty.	UC-05	`PolicyEngine` / replay job	`test_fr_28_fixed_target_reports_budget_and_uncertainty`	Behavior when N cannot be reached is **Open [Q5]**.
FR-29	In Suppression-only mode, the policy engine shall exclude customers classified as negative-uplift according to the agreed	UC-05	`PolicyEngine` / replay job	`test_fr_29_suppression_only_excludes_suppressed_customers`	Definition of "negative-uplift" and treatment of abstained customers are **Open [Q2]**. T32 proposes using the

ID	Requirement	Source use case	Realized by	Planned verification test	Notes / decision dependency
	suppression rule.				sleeping-dog suppression list and applying the confident-only eligible pool in every mode; this is not frozen.
FR-30	The policy engine shall place a customer on the sleeping-dog suppression list only when the estimated uplift is negative and the entire 95% interval is below zero; that customer shall not be contacted in any policy mode.	UC-05	`PolicyEngine` suppression-list construction	`test_fr_30_sleeping_dog_requires_entire_interval_below_zero`	The relation between this list and the point-estimate selection floor is explicitly open [Q2].
FR-31	Once allocation is fixed, the replay shall join the selected target list to the original recorded treatment assignment; selected targets originally assigned control shall form the replay holdout, and selected targets originally assigned treatment shall form the treated group. The replay shall not draw a new random holdout.	UC-05, UC-06	Replay job after allocation; recorded-arm join	`test_fr_31_replay_holdout_uses_recorded_assignment`	Decided by ADR 0002 and T52/#94. The assignment join must not leak treatment or outcome into pre-reveal scoring/allocation. Holdout size is reported with the result.
FR-32	The system shall report the allocation's contact count, expected incremental conversions with an interval, expected incremental profit, and a downloadable target list.	UC-05	Dashboard Tab 1 and allocation output	`test_fr_32_allocation_summary_contains_required_outputs`	Specification §40. The UI's interactive allocation path versus replay-config-only allocation is **Open [Q3(b)]**.
FR-33	Customers with either abstention verdict shall be reported separately and shall not be silently merged with the suppression list or a confident recommendation.	UC-04, UC-05	Allocation summary and export preparation	`test_fr_33_abstentions_separate_from_suppression`	Eligibility rules, the exported label for abstentions, and recommendation/segment fields are **Open [Q2]**. T32 proposes that abstained customers are not eligible in any policy mode.

2.5 Measurement and audit (UC-06)

ID	Requirement	Source use case	Realized by	Planned verification test	Notes / decision dependency
FR-34	After a valid reveal, the system shall measure each policy's incremental effect as the outcome rate in the selected targets recorded as treated minus the rate in the selected targets recorded as control.	UC-06	Replay measurement stage	`test_fr_34_measured_effect_uses_recorded_treatment_arms`	Decided replay semantics in ADR 0002/T52. T32 states holdout is the recorded control arm, not a newly randomized subset.
FR-35	The system shall compute a 95% percentile bootstrap interval using at least 1,000	UC-06	Replay measurement/bootstrap utility	`test_fr_35_bootstrap_resamples_within_arms`	Specification §28 and T32 glossary. The exact handling of too few control events

ID	Requirement	Source use case	Realized by	Planned verification test	Notes / decision dependency
	resamples drawn within each recorded treatment arm.				is represented in FR-36.
FR-36	If a measured policy or decile does not meet the configured minimum control-event count, the system shall report `insufficient_control_data` and shall not claim a measured effect for it.	UC-06	Measurement and result-serialization stage	`test_fr_36_measurement_abstains_with_too_few_control_events`	Recorded control is a subset of selected targets; this can occur frequently on Criteo [T32, Q10]. The threshold is configuration-driven.
FR-37	A measured replay result shall compare Uplift, Response Model, Blanket Contact and Random Contact at an identical budget and show contact count, total cost, predicted and measured incremental conversions with intervals, incremental profit, profit per unit spent, and cumulative curves with bootstrap bands.	UC-06	Replay comparison stage; results consumed by Dashboard Tabs 2 and 4	`test_fr_37_recorded_replay_compares_four_policies`	The uplift policy's mode comes from scenario configuration. Cost and margin must be stated with profit figures; do not describe estimates as money saved.
FR-38	The system shall produce predicted-versus-measured incremental-conversion calibration by decile and store the complete measurement artifacts with the scenario and MLflow run.	UC-06	Replay measurement, calibration output and MLflow tracking	`test_fr_38_calibration_and_measurement_artifacts_persisted`	K3/K5; include holdout size, curves, intervals, bootstrap seed and metric-library settings.
FR-39	The same replay implementation shall be used by the offline CLI job and `POST /policy/simulate`; for identical effective inputs and seed, the two paths shall produce equivalent result values.	UC-06, UC-07	Shared `netlift` replay implementation; CLI and `POST /policy/simulate` adapters	`test_fr_39_cli_and_simulate_return_same_result_for_same_inputs_and_seed`	Required by ADR 0002. Effective scenario/slice/model/seed binding is **Open [Q3(c)]**; deferred to T45 (#57).

2.6 Policy comparison what-if (UC-07)

ID	Requirement	Source use case	Realized by	Planned verification test	Notes / decision dependency
FR-40	`POST /policy/simulate` shall re-run the shared replay code for a what-if using supplied budget, cost and margin, rather than reading or modifying the stored result of UC-06; the call shall not create a recorded MLflow replay run.	UC-07	`POST /policy/simulate` and shared replay implementation	`test_fr_40_policy_simulate_reruns_without_mutating_recorded_run`	ADR 0002, decision 2. Which scenario or data slice, model version and seed the call uses beyond those three inputs is **Open [Q3(c)]**; deferred to T45 (#57).
FR-41	The Policy Comparison view shall show, for each of the four policies, number contacted, total cost, incremental conversions with a confidence interval, incremental profit and	UC-07	Dashboard Tab 2; `POST /policy/simulate` response	`test_fr_41_policy_comparison_table_and_assumptions`	Profit figures are estimates, not realised money (K3).

ID	Requirement	Source use case	Realized by	Planned verification test	Notes / decision dependency
	profit per unit spent, with cost and margin assumptions named alongside profit figures.				
FR-42	The Policy Comparison view shall draw cumulative-incremental curves with bootstrap bands and allow a visual budget marker to move across all displayed curves without a request solely to move the marker.	UC-07	Dashboard Tab 2 chart interaction	`test_fr_42_budget_marker_moves_across_all_curves`	As T32 describes, changing budget/cost/margin used to calculate the table triggers another `POST /policy/simulate` call.
FR-43	The comparison shall state in words whether the 95% interval for the uplift-versus-response-model difference excludes zero in favour of uplift; if the interval contains zero, it shall say the policies are not distinguishable on this data and shall name no winner.	UC-07	Policy-comparison summary	`test_fr_43_policy_difference_interpretation`	K1, specification §28. Policy-level abstention styling details remain open [Q2(d)].
FR-44	A simulation request shall not mutate the measured scenario state or stored UC-06 result. **Proposed [Q3]:** the comparison should not start before the scenario has a measured recorded result.	UC-07	`POST /policy/simulate` guard and read-only result handling	`test_fr_44_simulate_requires_measured_scenario_and_is_read_only`	The measured-scenario precondition and binding to a specific scenario are **Proposed [Q3]**. Request fields identifying the scenario/data slice, model version and seed are **Open [Q3(c)]**; deferred to T45 (#57). The request must have a defined identity for its effective inputs before implementation.
FR-45	Until Q1 is decided, replays shall use only the validation split. The locked test split may be opened only through `LockedTestSplit.open(reason)` under the T70 procedure. A simulation shall not re-open the locked test split; if the selected input would require a new read, the call shall be refused and only already recorded UC-06 figures may be shown.	UC-07	Replay split guard / `LockedTestSplit` access policy	`test_fr_45_simulate_refuses_locked_test_split_reuse`	Interim rule in ADR 0002 (commit 700e455): validation split only until Q1 is decided; locked test access only through `LockedTestSplit.open(reason)` under T70. Q1 remains open for the final stage-specific split policy.

2.7 Drift monitoring (UC-09)

ID	Requirement	Source use case	Realized by	Planned verification test	Notes / decision dependency
FR-46	`GET /monitor/drift` shall return the latest available drift report, including per-feature PSI or KS against the training reference, uplift-score distribution drift, and	UC-09	Drift-report computation; `GET /monitor/drift`; Dashboard Tab 4	`test_fr_46_drift_report_contains_feature_score_and_quality_status`	Alert thresholds are read from configuration, specification §32. Behavior when no report exists is unspecified in T32 and must be clarified before the API

ID	Requirement	Source use case	Realized by	Planned verification test	Notes / decision dependency
	data-quality status for null rates, range violations and schema changes.				contract freezes.
FR-47	The Monitoring view shall display the registered model version, training date, and results of simulated campaign replays; any effect-drift panel shall be explicitly labelled as mocked from replay output.	UC-09	Dashboard Tab 4	`test_fr_47_monitoring_labels_mocked_effect_drift`	Feature/score drift is built this semester; real effect drift requiring a permanent randomized holdout and real outcome window is deployment-only.
FR-48	This-semester drift reports shall be producible on demand from a scored batch and training reference; scheduled drift-report generation shall be treated as deployment-only.	UC-09	On-demand drift report workflow; optional future Scheduler integration	`test_fr_48_drift_report_can_be_generated_on_demand`	The on-demand/current-semester split is **Proposed [Q7]** and needs confirmation.

2.8 Retraining (UC-10)

ID	Requirement	Source use case	Realized by	Planned verification test	Notes / decision dependency
FR-49	A Data Scientist shall be able to start a retraining run as an offline pipeline job; the job shall be reproducible from configuration and a git commit hash.	UC-10	Offline training pipeline; MLflow tracking and Model Registry	`test_fr_49_retraining_pipeline_runs_from_config_and_commit`	Retraining is not exposed as a new API endpoint. Scheduled triggering is deployment-only (ADR 0002).
FR-50	The retraining pipeline shall validate schema, audit leakage, create the specified joint stratified split, fit baselines before uplift estimators, and evaluate candidates with bootstrap intervals.	UC-10	Training pipeline stages	`test_fr_50_retraining_runs_required_validation_and_evaluation_stages`	Specification §23 and §28; exact split restrictions must align with Q1/T70.
FR-51	The retraining pipeline shall fail if a forbidden column reaches the model matrix and shall fail the candidate's K4 gate if the 95% interval of uplift under the permuted-treatment negative control does not contain zero.	UC-10	Leakage audit and negative-control evaluation gates	`test_fr_51_retraining_blocks_leakage_and_negative_control_failure`	K4/K7.
FR-52	A completed candidate shall be registered as a new model version with its evaluation attached, while the previous model version remains registered and available for rollback.	UC-10	MLflow Model Registry lifecycle	`test_fr_52_candidate_registered_with_evaluation_and_previous_retained`	Whether and when a candidate becomes the new champion/production version is **Open [Q9]**; this requirement does not authorize automatic promotion.
FR-53	The Data Scientist shall be able to start the pipeline manually in this semester; automatic cadence/drift-triggered retraining shall be	UC-10	Manual pipeline entry point; future Scheduler integration	`test_fr_53_retraining_can_be_started_manually`	The OR-vs-AND interpretation of cadence/drift triggers is **Proposed [Q9]** and is not frozen by ADR 0002.

ID	Requirement	Source use case	Realized by	Planned verification test	Notes / decision dependency
	marked deployment-only.				

3. Use-case coverage

T32 use case	Requirement IDs
UC-01 Define a replay scenario	FR-01–FR-04
UC-02 Score a batch of customers	FR-08–FR-18
UC-03 Review model evidence	FR-19–FR-20
UC-04 Deliver the target list to a campaign	FR-25, FR-33
UC-05 Allocate under budget	FR-26–FR-33
UC-06 Reveal outcomes and measure	FR-05–FR-07, FR-31, FR-34–FR-39
UC-07 Compare policies	FR-39–FR-45
UC-08 Inspect a customer explanation	FR-21–FR-24
UC-09 Review drift	FR-46–FR-48
UC-10 Retrain the model	FR-49–FR-53

Coverage check: every use case has at least one requirement and every FR has a source use case. The requirement/test names above are **planned names**. As tests are implemented, retain the FR identifier in the test name or test docstring so traceability remains bidirectional.

4. API and interface boundary

Interface / execution path	Requirements served	Boundary
`POST /score`	FR-08–FR-18, FR-21–FR-24	Single-customer scoring; no extra inputs/fields beyond the frozen API contract until Q4 is resolved.
`POST /score/batch`	FR-08–FR-18	Batch scoring and summary.
`POST /policy/simulate`	FR-39–FR-45	What-if re-simulation through shared replay code; no recorded replay run. Scenario/slice/model/seed binding is **Open [Q3(c)]**; deferred to T45 (#57).
`GET /model/info`	FR-19	Registered model version, training date, metrics and intervals.
`GET /monitor/drift`	FR-46–FR-48	Latest drift report; empty-report behavior needs clarification.
`GET /health`	FR-16	Liveness/readiness.
Offline replay CLI job	FR-01–FR-07, FR-31, FR-34–FR-39	The recorded UC-06 run; configuration defines scenario; result logged to MLflow.
Offline training pipeline	FR-49–FR-53	UC-10; no retraining endpoint.
Dashboard Tab 1	FR-15, FR-25–FR-33	Scoring result, allocation view, target-list download. Whether interactive allocation

Interface / execution path	Requirements served	Boundary
		calls the decision layer remains open [Q3].
Dashboard Tab 2	FR-37, FR-41–FR-45, FR-47	Recorded comparisons plus interactive simulation. Dashboard access to stored replay results (MLflow directly or an existing API path) remains open for T45/T62.
Dashboard Tab 3	FR-19–FR-20	Model evidence.
Dashboard Tab 4	FR-38, FR-47–FR-48	Recorded results, feature/score drift, and explicitly mocked effect drift.
Campaign Execution System	None in current-semester scope	Live delivery, permanent holdout, outcome ingestion and live-effect measurement are deployment-only.

5. Open decisions required before freezing affected requirements

Question	Affected requirements	Needed decision / handling
Q1: replay split and locked-test access	FR-01, FR-45, FR-50	Q1 remains open for the final stage-specific split policy. Interim rule in ADR 0002 (commit 700e455): until Q1 is decided, replays use only the validation split; the locked test split may be opened only through `LockedTestSplit.open(reason)` under the T70 procedure. Do not implement repeated locked-test reads as a default.
Q2: abstention, recommendation, segment and allocation behavior	FR-13–FR-15, FR-22, FR-24, FR-26–FR-30, FR-33, FR-36, FR-43	Decide recommendation/segment values under abstention, whether abstained customers are ineligible in every mode, the difference between a non-positive selection floor and sleeping-dog suppression, how `k` is handled when the eligible pool is small, and policy-level abstention.
Q3: execution and API/UI boundary	FR-05, FR-25, FR-32, FR-39–FR-44	Name who starts the CLI replay; decide whether Tab 1 supports interactive allocation; define which scenario or data slice, model version and seed identify a `POST /policy/simulate` request (**Open [Q3(c)]**; deferred to T45 (#57)); decide how Tabs 2/4 access recorded MLflow results. Keep the API to six endpoints per ADR 0002.
Q4: cost/margin source for standalone scoring	FR-17	Specify whether cost and margin are request inputs or configuration values before freezing `expected_incremental_profit` for standalone scoring.
Q5: unreachable fixed target	FR-28	Define response when no set can reasonably deliver target `N`.
Q7: scheduler status	FR-48, FR-53 and scheduled paths in FR-08 / FR-46	Confirm that scheduled batch scoring, scheduled drift reporting and scheduled retraining are deployment-only and that semester drift reports are on demand.
Q9: retraining trigger and promotion	FR-49, FR-52–FR-53	Confirm cadence OR drift threshold versus another rule; define promotion gate/locked-test procedure. ADR 0002 leaves promotion open and does not authorize automatic promotion.
Q11: K2 success rule	FR-20	Align specification §46 with `config/kpis.yaml`; the K2 interval must be entirely above zero, not merely exclude zero. PR #94 remains open; verify the merged specification and close Q11 after #94 merges.
Q12: scenario policy defaults	FR-02	Confirm default Fixed budget and reject Fixed target without `N`, or document the selected alternative.

6. Source and consistency notes

- T32's use-case IDs are frozen. This document does not renumber or rename UC-01–UC-10.
- Recorded replay holdout means the selected targets originally assigned to control. It is not a newly randomized holdout. Measured effect is treated rate minus control rate in the selected targets, with within-arm bootstrap resampling.
- The replay and retraining jobs are offline execution paths, not new API endpoints. `POST /policy/simulate` re-runs shared replay code and does not reuse the stored UC-06 result.
- This document does not claim that planned tests are implemented, that deployment-only flows are built, or that proposals marked [Q#] are approved.

4. Non-functional Requirements

Author: Aliaa Gomaa · Task: T34 · Reviewed by: Mazen

Every requirement states how it is measured. Targets come from `config/kpis.yaml` where one exists; otherwise the target is “measured and reported in M5”. No invented thresholds.

ID	Category	Requirement	How it is measured	Target
NFR-01	Reproducibility	A clean checkout plus one documented command reproduces the headline result.	Run the command on a fresh environment and compare the output with the recorded result.	Same result as recorded: reproduces exactly, from config plus the git commit hash (K8).
NFR-02	Performance	Scoring a single customer returns quickly, and scoring a large batch completes in reasonable time.	Load test against the deployed endpoint: p95 latency of a single score, and the time for a one-million-row batch on the project’s own hardware.	p95 under 300 ms for a single score; a one-million-row batch inside ten minutes (K9). Team-set targets, to be revised once first measured.
NFR-03	Auditability	Every score is logged together with the version of the model that produced it (`model_version`).	Check that every logged score record contains `model_version`.	Measured and reported in M5.
NFR-04	Correctness	No recommendation is produced without a `verdict` field, and every reported metric comes with a confidence interval.	Automated check on scoring responses: every recommendation comes with a non-empty `verdict`. Check on report outputs: each metric is paired with a confidence interval.	Metrics: reported with a confidence interval (K1, K2, K3). Verdict field: measured and reported in M5.
NFR-05	Security	Secrets are stored only in `.env`, never in code, config or the repository, and the system stores and processes no personal data.	Secret scan of the repository and a check that `.env` is git-ignored. Review of datasets, logs and outputs for personal identifiers (both datasets are public and de-identified).	Measured and reported in M5.
NFR-06	Usability	The abstention verdicts (`insufficient_evidence`, `insufficient_control_data`) are visually distinct from a confident recommendation.	Visual review of the dashboard, with an abstention and a confident recommendation side by side.	Measured and reported in M5.
NFR-07	Reliability	The system handles bad or missing input and repeated runs without crashing or silently producing wrong output.	Failed runs and unhandled errors counted across test runs and CI.	Measured and reported in M5.
NFR-08	Maintainability	Every change is verified by the CI gates before it merges.	CI on every pull request: lint, type check, unit, data, model-behaviour, integration and API tests.	Green on the merge commit of every pull request (K10).